Question 1: Django Introduction & Architecture

1. What is Django and why is it used?

Django is a high-level web framework written in Python. It provides a ready-made structure and a collection of tools for building websites and web applications. Instead of writing every common feature from the beginning, a developer can use Django's built-in components for tasks such as handling web requests, working with a database, managing users and displaying pages. It is used because it helps developers build applications in an organized, maintainable and secure way, while reducing the amount of repetitive code.

2. Five important features of Django

- **Object-Relational Mapper (ORM):** Django lets developers work with database records through Python classes and objects, generating many database queries automatically.
- **Built-in administration site:** Django can create an admin interface where trusted staff can add, view, update and delete application data.
- **URL routing:** Its URL dispatcher connects a requested address to the Python view that handles it, keeping page routing organized.
- **Template system:** Django templates combine HTML with dynamic values and template tags, keeping page markup separate from Python logic.
- **Security tools:** Django includes protections against common risks such as CSRF, SQL injection and cross-site scripting (XSS). Developers must still use it correctly.

3. Difference between MVC and MVT architecture

MVC stands for Model-View-Controller. It separates an application into three main parts: the Model manages data, the View is responsible for the user-facing presentation, and the Controller handles incoming input and coordinates what happens next.

MVT stands for Model-View-Template, which is the pattern used by Django. The Model manages data, the View contains the request-handling logic, and the Template controls how the page is presented. Django's URL dispatcher routes a request to the correct View. Therefore, Django's View performs much of the coordinating role commonly associated with the MVC Controller, while Django's Template fills the presentation role commonly associated with the MVC View. The names are different, but both patterns aim to separate responsibilities.

4. Roles of Model, View, Template and URL in Django

- **Model:** Defines the application's data structure as a Python class that Django's ORM uses to work with database records.
- **View:** Handles a request, runs application logic, works with model data when needed, and returns a response.
- **Template:** Provides the page layout, usually in HTML, and displays dynamic information supplied by the view.
- **URL:** A URL pattern matches a requested address and directs it to the appropriate view.

Question 2: Django Project Setup

The following steps show how to create and run a basic Django project. The commands below use Windows PowerShell. Make sure Python is installed and available from the terminal before starting.

Step 1: Create a project folder and open it in the terminal.

Choose a location for the project, create a folder and move into it. For example:

```
mkdir django-learning
cd django-learning
```

Step 2: Create a virtual environment.

A virtual environment keeps this project's Python packages separate from packages used by other projects. Create it with:

```
py -m venv venv
```

Step 3: Activate the virtual environment.

Activate it in Windows PowerShell with:

```
.\venv\Scripts\Activate.ps1
```

Step 4: Install Django.

With the environment active, install Django using pip:

```
python -m pip install django
```

Step 5: Create the Django project.

Run the following command to create a project named myproject:

```
django-admin startproject myproject
```

Step 6: Move into the project directory.

Enter the new folder, where Django created the project files:

```
cd myproject
```

Step 7: Run the development server.

Start Django's local development server with:

```
python manage.py runserver
```

Other operating systems

On macOS or Linux, the virtual-environment creation command is usually python3 -m venv venv, and activation is source venv/bin/activate. The Django installation and project commands are otherwise the same, using the Python command available on that system.

Purpose of the main Django files

File	Purpose
manage.py	A command-line utility for this project. It is used to run development commands such as starting the server, applying database migrations and creating an administrator account.
settings.py	Contains the project's configuration, including installed applications, middleware, database settings, templates, language and other options.
urls.py	Contains URL patterns. Django checks these patterns to decide which view should handle an incoming URL.
wsgi.py	Provides the WSGI application entry point, which lets a WSGI-compatible web server communicate with the Django project when it is deployed.

Accessing the project

After the development server starts, Django displays a local address in the terminal. Open http://127.0.0.1:8000/ in a web browser to view the project. The development server is intended for local development and testing, not for serving a production website.

Conclusion

Django provides a structured way to develop Python web applications. Its MVT design separates data, request-handling logic and presentation, while the setup process creates an isolated environment and a project that can be run locally.